

Engenharia de Software II

André L. D. Rossi

Universidade Estadual Paulista “Júlio de Mesquita Filho” (UNESP)
Faculdade de Ciências (FC) / Departamento de Computação (DCo)
Bauru, SP - Brasil

1. Panorama Geral
2. Projeto de Componentes
3. Condução de projetos de componentes
4. Desenvolvimento baseado em componentes

Panorama Geral

O que é:

- Um conjunto completo de componentes de software é definido durante o projeto da arquitetura.
- Porém, os detalhes de processamento e estruturas de dados internas de cada componente não são representados em um nível de abstração próximo ao código.
- O projeto de componentes define as estruturas de dados, os algoritmos, as características das interfaces e os mecanismos de comunicação alocados a cada componente de software.

Por que é importante?

- Permite revisar os detalhes do projeto em termos de correção e consistência com outras representações:
 - Interfaces
 - Arquitetura de dados
 - ...

Qual é o artefato?

O projeto para cada componente, representado em notação gráfica, tabular ou baseada em texto, é o principal artefato durante o projeto de componentes.

Projeto de Componentes

Projeto de Componentes

- O projeto de componentes ocorre depois que a primeira iteração do projeto da arquitetura tiver sido concluída.
- Nesse estágio, a estrutura geral dos dados e programas do software já foi estabelecida.
- O intuito é transformar o modelo de projeto em software operacional.
- Porém, o nível de abstração do modelo de projetos existente é relativamente alto, e o nível de abstração do programa operacional é baixo.

O que é componente?

- Componente é um bloco construtivo modular para software de computador.
- Mais formalmente, a UML define componente como:
*[...] uma parte modular, possível de ser implantada e **substituível** de um sistema que encapsula implementação e expõe um conjunto de interfaces.*

O que é componente?

- Os componentes devem se comunicar e colaborar com **outros componentes** e entidades externas:
 - Outros sistemas
 - Dispositivos
 - Pessoas
- O verdadeiro significado do termo componente dependerá do ponto de vista do engenheiro de software que o utiliza.

Uma visão orientada a objetos

Uma visão orientada a objetos

- No contexto da engenharia de software orientada a objetos, um componente contém um conjunto de classes colaborativas.
- Também precisam ser definidas todas as interfaces que permitem que as classes se comuniquem e colaborem com outras classes de projeto.

Uma visão orientada a objetos

- Por exemplo, considere um software a ser criado para uma gráfica.
- O objetivo geral do software é coletar os requisitos do cliente na recepção da loja, orçar um trabalho e, em seguida, passar a tarefa para um centro de produção automatizado.
- Durante a engenharia de requisitos, foi obtida uma classe de análise denominada ServiçoDeImpressão (PrintJob)

Uma visão orientada a objetos

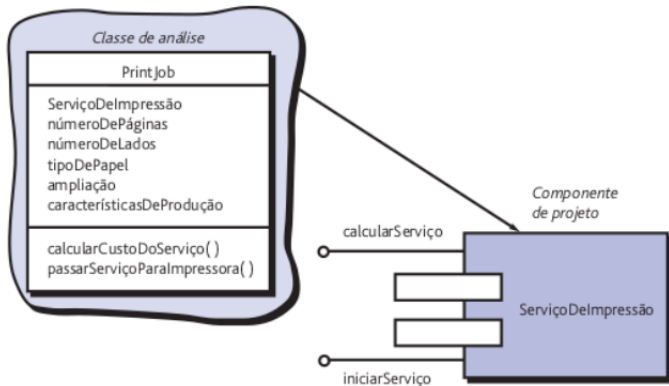


Figura 1: Classe de análise e componente.

Uma visão orientada a objetos

- Os detalhes do componente *ServiçoDeImpressão* devem ser elaborados a fim de fornecer informações suficientes para orientar a implementação.
- A classe de análise original é detalhada para implementar a classe na forma do componente *ServiçoDeImpressão*.

Uma visão orientada a objetos

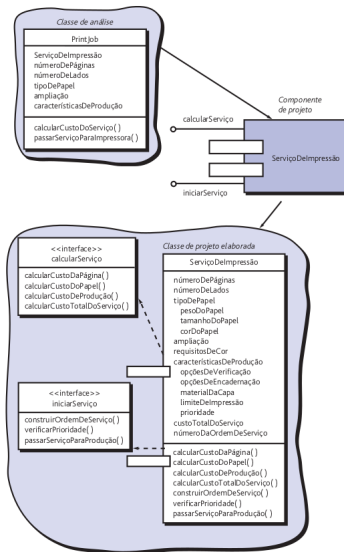


Figura 2: Classe de projeto para o componente ServiçoDeImpressão.

Projeto de componentes baseados em classes

Na abordagem de engenharia de software orientada a objetos, o projeto de componentes se concentra na elaboração de classes contidas no modelo de requisitos/arquitetura:

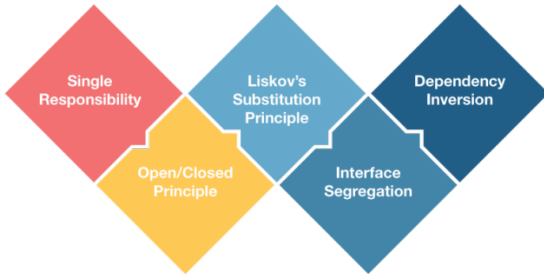
- Classes específicas do domínio do problema
- Classes de infraestrutura, que dão suporte a serviços

A descrição detalhada dos atributos, operações e interfaces utilizados por essas classes é o detalhe de projeto exigido como precursor da atividade de construção.

Cinco princípios básicos aplicáveis ao projeto de componentes têm sido amplamente adotados quando se aplica à ES orientada a objetos.

Princípios básicos de projeto

S.O.L.I.D.



Princípios básicos de projeto

- Motivação é criar projetos mais fáceis de **modificar**.
 - Isso inclui reduzir a propagação de efeitos colaterais na ocorrência de modificações.
- Podemos usar tais princípios como guias, à medida que cada componente de software é desenvolvido.

Princípios básicos de projeto

A Tabela 1 a seguir mostra as propriedades contempladas ao seguir cada um desses princípios.

Princípio de Projeto	Propriedade de Projeto
(S) Responsabilidade Única	Coesão
(O) Aberto/Fechado	Extensibilidade
(L) Substituição de Liskov	Extensibilidade
(I) Segregação de Interfaces	Coesão
(D) Inversão de Dependência	Acoplamento

Tabela 1: Os cinco princípios de projeto denominados de SOLID.

Princípio da Responsabilidade Única

Princípio da Responsabilidade Única

“Toda classe deve ter uma única responsabilidade!”

- Esse princípio é uma aplicação direta da ideia de coesão.
- Um corolário desse princípio recomenda separar **apresentação** de **regras de negócio**, pois as classes de apresentação e as classes de regras de negócio podem evoluir e sofrer modificações por razões distintas.
- Quem implementa as classes de apresentação? E quem implementa as classes de regras de negócio?

Princípio da Responsabilidade Única

```
public class Employee {  
  
    private String fullName;  
    private String dateOfJoining;  
    private String annualSalaryPackage;  
  
    // standard getters and setters methods  
  
    public long calculateEmployeeSalary(Employee emp) { ... }  
    public long calculateEmployeeLeaves(Employee emp) { ... }  
    public long calculateTaxOnSalary(Employee emp) { ... }  
  
    public Employee saveEmployee(Employee emp) { ... }  
    public Employee updateEmployee(Employee emp) { ... }  
}
```


Princípio da Responsabilidade Única

```
class Animal:
    def __init__(self, nome, pernas, quantidade, imagem):
        self.nome = nome
        self.pernas = pernas
        self.quantidade = quantidade
        self.imagem = imagem
    def calcular_pernas(self):
        return f'Número de pernas em {self.nome} é: ', self.pernas * self.
            quantidade

    def calcular_olhos(self):
        return f'Número de olhos em {self.nome} é: ', self.quantidade * 2

    def alterar_tamanho_imagem(self):
        # Este método altera o tamanho da imagem do animal
        pass

    def rotacionar_imagem(self):
        # Este método rotaciona a imagem do animal
        pass
```

Princípio da Responsabilidade Única

Exercício de refatoração:

```
public String statement() {
    double totalAmount = 0;
    int frequentRenterPoints = 0;
    Enumeration rentals = _rentals.elements();
    String result = 'Rental Record for ' + getName() + '\n';
    while (rentals.hasMoreElements()) {
        double thisAmount = 0;
        Rental each = (Rental) rentals.nextElement();

        //determine amounts for each line
        switch (each.getMovie().getPriceCode()) {
            case Movie.REGULAR:
                thisAmount += 2;
                if (each.getDaysRented() > 2)
                    thisAmount += (each.getDaysRented() - 2) * 1.5;
                break;
            ...
        }
    }
}
```

Princípio da Segregação de Interfaces (Interface Segregation Principle -ISP)

Princípio da Segregação de Interfaces

- É melhor usar várias interfaces específicas do cliente do que uma única interface de propósito geral.
- Há diversas ocasiões em que componentes para vários clientes usam uma operação fornecida por uma classe-servidora.
- O ISP sugere a criação de uma interface especializada para atender cada categoria principal de clientes.
- O objetivo é evitar que clientes dependam de interfaces com métodos que eles não vão usar.
- O ISP é um caso particular do princípio SRP.

Princípio da Segregação de Interfaces

Exemplo de interface que viola o ISP:

```
interface Funcionario {  
    double getSalario();  
    double getFGTS();  
    int getSIAPE();  
    ...  
}
```

Por que o ISP é violado nesse caso?

Princípio da Segregação de Interfaces

```
public interface Vehicle{
    void accelerate();
    void applyBrakes();
    void fly();
}

public class Bus implements Vehicle{
    @Override
    public void accelerate() {...}

    @Override
    public void applyBrakes() {...}

    @Override
    public void fly(){
        // dummy implementation
    }
}
```

```
public class Aeroplane implements
    Vehicle{
    @Override
    public void accelerate() {...}

    @Override
    public void applyBrakes() {...}

    @Override
    public void fly(){...}
}
```

Princípio da Segregação de Interfaces

Sugestões para resolver o problema e não violar o princípio ISP?

Princípio da Inversão da Dependência (Dependency Inversion Principle - DIP)

Princípio da Inversão da Dependência

- “Dependa de abstrações. Não dependa de concretizações”.
- Abstrações (interfaces) são mais estáveis do que implementações concretas (classes).
- Quanto mais um componente depender de outros componentes concretos (e não de abstrações), mais difícil será estendê-lo.

Princípio da Inversão da Dependência

Exemplo 1:

```
interface I { ... }

class C1 implements I {
    ...
}
class C2 implements I {
    ...
}

class Cliente {
    I i;
    Cliente (I i) {
        this.i = i;
        ...
    } ...
}
```

Princípio da Inversão da Dependência

Exemplo 1:

```
interface I { ... }

class C1 implements I {
    ...
}

class C2 implements I {
    ...
}

class Cliente {
    I i;
    Cliente (I i) {
        this.i = i;
        ...
    } ...
}
```

```
class Main {
    void main () {
        C1 c1 = new C1();
        new Cliente(c1);
        ...
        C2 c2 = new C2();
        new Cliente(c2);
        ...
    }
}
```

Princípio da Inversão da Dependência

Exemplo 2 - viola o DIP:

```
class SQLRepository {  
    public void save() {  
        ...  
    }  
}  
  
class NoSQLRepository {  
    public void save() {  
        ...  
    }  
}  
  
public class Service {  
    private SQLRepository repository = new SQLRepository();  
  
    public void save() {  
        repository.save();  
    }  
}
```

Princípio do Aberto-Fechado

- “Um módulo [componente] deve ser aberto para a extensão, mas fechado para modificações”.
- Essa afirmação pode parecer uma contradição, mas representa uma das características mais importantes de um bom projeto de componentes.
- Em outras palavras, devemos especificar o componente para permitir que ele seja estendido (em seu domínio funcional) sem a necessidade de fazer modificações internas (em nível de código ou lógica) no próprio componente.
- Para tanto, criamos abstrações que servem como um divisor entre a funcionalidade que provavelmente será estendida e a classe de projeto em si.

Princípio do Aberto-Fechado (OCP, Open-Closed Principle)

Por exemplo, suponhamos que a função de segurança do CasaSegura faça uso de uma classe Detector que deve verificar o estado de cada tipo de sensor de segurança.

É provável que, à medida que o tempo for passando, o número e os tipos de sensores de segurança cresçam.

Se a lógica de processamento interna for implementada como uma sequência de construções se-então-senão (ou *switch*), cada uma delas tratando de um tipo de sensor diferente, a adição de um novo tipo de sensor exigirá lógica de processamento interna adicional.

Isso é uma violação do OCP.

Princípio do Aberto-Fechado

A interface sensor apresenta uma visão consistente dos sensores para o componente detector. Se for adicionado um novo tipo de sensor, nenhuma mudança será necessária na classe Detector (componente). O OCP é preservado.

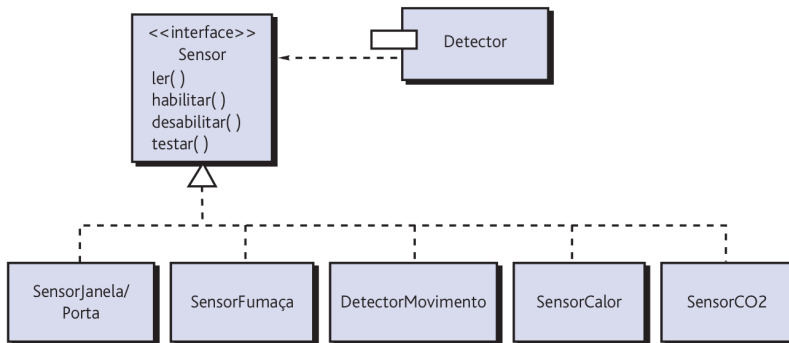


Figura 3: Seguindo o OCP.

Princípio do Aberto-Fechado

```
public class EmployeeSalary {  
  
    public Long calculateSalary(Employee emp) {  
        Long salary = null;  
  
        if (emp.getType().equals("PERMANENT")) {  
  
            salary = (totalWorkingDay * basicPay) + getCompanyBenefits() + getBonus();  
        } else if (emp.getType().equals("CONTRACT")) {  
  
            salary = (totalWorkingDay * basicPay);  
        }  
        return salary;  
    }  
}
```


Princípio do Aberto-Fechado

A classe Collections de Java segue o Princípio Aberto/Fechado.

```
List<String> nomes;  
nomes = Arrays.asList('joao', 'maria', 'alexandre', 'ze');  
Collections.sort(nomes);  
  
System.out.println(nomes);  
// resultado: ['alexandre', 'joao', 'maria', 'ze']
```

Princípio do Aberto-Fechado

Se for necessário usar o sorte para ordenar as strings de acordo com o tamanho em caracteres, só preciso implementar um objeto Comparator.

```
Comparator<String> comparador = new Comparator<String>() {  
    public int compare(String s1, String s2) {  
        return s1.length() - s2.length();  
    }  
};  
Collections.sort(nomes, comparador);  
  
System.out.println(nomes);  
// resultado: [ze, joao, maria, alexandre]
```

Princípio da Substituição de Liskov

- “As subclasses devem ser substitutas de suas classes-base”.
- Esse princípio, originalmente proposto por Barbara Liskov, explicita regras para redefinição de métodos de classes base em classes filhas.
- Com isso, um componente que usa uma classe-base deve continuar a funcionar apropriadamente caso uma classe derivada da classe-base seja passada para o componente em seu lugar.

Princípio da Substituição de Liskov

O LSP exige que qualquer classe derivada de uma classe-base deve honrar qualquer contrato implícito entre a classe-base e os componentes que a utilizam.

No contexto desta discussão, um “contrato” é uma pré-condição que deve ser verdadeira antes de o componente usar uma classe-base e uma pós-condição que deve ser verdadeira após o componente usar uma classe-base.

Ao criar classes derivadas, certifique-se de que atendem às pré-condições e às pós-condições.

Princípio da Substituição de Liskov

Exemplo que viola (fortemente) o LSP:

```
class A {  
    int soma(int a, int b) {  
        return a+b;  
    }  
}  
  
class B extends A {  
    int soma(int a, int b) {  
        String r = String.valueOf(a)  
            + String.valueOf(b);  
        return Integer.parseInt(r);  
    }  
}
```

```
class Cliente {  
    void f(A a) {  
        // pode retornar 3 ou 12  
        a.soma(1,2);  
        ...  
    }  
}  
  
class Main {  
  
    void main() {  
        A a = new A();  
        B b = new B();  
        Cliente cli = new Cliente();  
        cli.f(a);  
        cli.f(b);  
    }  
}
```

Princípio da Inversão da Dependência

```
public class Car {  
    public void fuel() { ...}  
    public void wheels() { ... }  
    public void run() { ... }  
}
```

```
public class TeslaToyCar extends Car {  
    @Override  
    public void fuel() {  
        throw new IllegalStateException('Not  
            Supported');  
    }  
    @Override  
    public void run() {...}  
    @Override  
    public void wheels() {...}  
}
```

```
public class TeslaRealCar extends  
    Car {  
  
    @Override  
    public void fuel() {...}  
  
    @Override  
    public void run() {...}  
  
    @Override  
    public void wheels() {...}  
}
```

Princípios de empacotamento aplicáveis ao projeto de componentes

Princípios de empacotamento aplicáveis ao projeto de componentes

- Embora os princípios de projeto de componentes sejam úteis em termos de orientação, os componentes em si não vivem de forma isolada.
- Em muitos casos, componentes ou classes individuais são organizados em subsistemas ou pacotes.
- Faz sentido perguntar como deve ocorrer a atividade de empacotamento.
- Exatamente de que forma os componentes devem ser organizados à medida que o projeto prossegue?

Princípio da Equivalência de Reutilização de Versões (REP, Release Reuse Equivalency Principle)

- “A granularidade da reutilização é a granularidade da versão”.
- É estabelecido um contrato implícito entre o desenvolvedor da entidade reutilizável e quem vai usá-la.
- O desenvolvedor se compromete a estabelecer um sistema de controle de versões que ofereça **suporte e manutenção para as versões mais antigas** da entidade, enquanto os usuários vão atualizando gradualmente para a versão mais recente.

Princípio do Fechamento Comum (CCP, Common Closure Principle)

- “Classes que mudam juntas, devem ficar juntas” .
- As classes devem ser empacotadas de forma coesa.
- Ao serem empacotadas como parte de um projeto, devem tratar da mesma área funcional ou comportamental.
- Quando alguma característica dessa área tiver de mudar, é provável que apenas as classes contidas no pacote precisem ser modificadas.
-
- Isso leva a um controle de mudanças e gerenciamento de versões mais eficiente.

Princípio Comum da Reutilização (CRP, Common Reuse Principle)

- “As classes que não são reutilizadas juntas não devem ser agrupadas juntas”.
- Quando uma ou mais classes com um pacote muda(m), o número da versão do pacote muda.
- Se as classes não forem agrupadas de forma coesa, é possível que uma classe sem nenhuma relação com as demais contidas em um pacote seja alterada.
- Por essa razão, apenas as classes reutilizadas juntas devem ser incluídas em um pacote.

Coesão de Componentes: o diagrama de tensão

- Observe que os três princípios de coesão lutam entre eles:
 - REP e CCP são princípios que tendem a gerar componentes maiores
 - O CRP é um princípio que tende a gerar componentes menores
- Por isso, ao escolher entre os princípios, sempre haverá perdas.

Coesão de Componentes: o diagrama de tensão



Figura 4: Diagrama de tensão: como os três princípios de coesão interagem. As arestas indicam o “custo” de abandonar o princípio no vértice oposto [1].

Condução de projetos de componentes

Condução de projetos de componentes

Temos de transformar informações de modelos de arquitetura e requisitos em uma representação de projeto que nos dê detalhes suficientes para orientar a atividade da construção (codificação e testes).

Tais etapas representam um conjunto de tarefas típicas para um projeto de componentes – quando ele é aplicado a um sistema orientado a objetos.

“Se eu tivesse mais tempo, teria escrito uma carta mais curta.” Blaise Pascal

“One of my most productive days was throwing away 1000 lines of code.” (atribuído a Ken Thompson no livro: The Art of UNIX Programming, de Eric S. Raymond, p. 24.)

Condução de projetos de componentes

- **Etapa 1:** Identificar todas as classes de projeto correspondentes ao domínio do problema.
- **Etapa 2:** Identificar todas as classes de projeto correspondentes ao domínio de infraestrutura. Exemplos: GUI (muitas vezes disponíveis na forma de componentes reutilizáveis), componentes de administração de dados, etc.
- **Etapa 3:** Elaborar todas as classes de projeto que não são obtidas como componentes reutilizáveis.
 - **Etapa 3a:** Especificar detalhes de mensagens quando classes ou componentes colaboram entre si.
 - **Etapa 3b:** Identificar interfaces adequadas para cada componente.
 - **Etapa 3c:** Elaborar atributos e definir tipos de dados e estruturas de dados necessárias para implementá-los.
 - **Etapa 3d:** Descrever detalhadamente o fluxo de processamento contido em cada operação.

Condução de projetos de componentes

- **Etapa 4:** Descrever fontes de dados persistentes (bancos de dados e arquivos) e identificar as classes necessárias para gerenciá-los.
- **Etapa 5:** Desenvolver e elaborar representações comportamentais para uma classe ou componente.
- **Etapa 6:** Elaborar diagramas de implantação para fornecer detalhes de implementação adicionais.
- **Etapa 7:** Refatorar toda representação de projetos de componentes e sempre considerar alternativas.

Desenvolvimento baseado em componentes

Desenvolvimento baseado em componentes

No contexto da engenharia de software, a reutilização é uma ideia ao mesmo tempo antiga e nova.

Os programadores têm reutilizado ideias, abstrações e processos desde os primórdios da computação, mas a abordagem inicial para a reutilização era improvisada.

Hoje em dia, sistemas computacionais complexos e de alta qualidade devem ser construídos em prazos muito curtos e exigem uma abordagem mais organizada para a reutilização.

A engenharia de software baseada em componentes (CBSE) é um processo que enfatiza o projeto e a construção de sistemas baseados em computadores usando “componentes” de software reutilizáveis.

Desenvolvimento baseado em componentes

Considerando essa descrição, surge uma série de questões:

- É possível construir sistemas complexos montando-os por meio de um catálogo de componentes de software reutilizáveis?
- Isso pode ser realizado de maneira eficaz em termos de custo e tempo?
- Podem ser estabelecidos incentivos apropriados para estimular os engenheiros de software à reutilização em vez de reinventar?
- A gerência está disposta a incorrer na despesa adicional associada à criação de componentes de software reutilizáveis?
- A biblioteca de componentes necessários para a reutilização pode ser criada para torná-la acessível aos que precisam dela?
- Os componentes existentes podem ser encontrados por aqueles que precisam deles?

Cada vez mais, a resposta a cada uma dessas questões é “sim”.

Desenvolvimento baseado em componentes

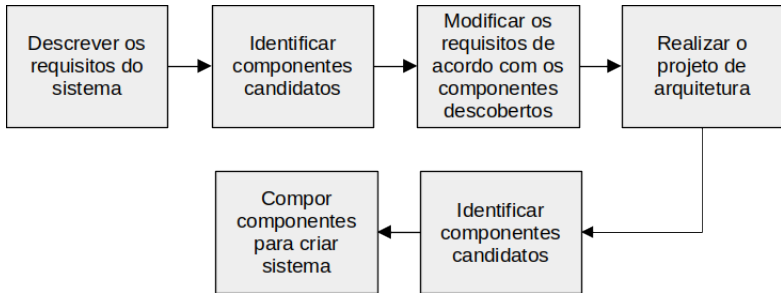


Figura 5: Etapas principais do CBSE. Cada componente possui suas próprias pré e pós-condições.

A CBSE pode ter as seguintes vantagens:

- Tempo de ciclo reduzido;
- Divisão dos custos do desenvolvimento dos componentes entre múltiplos projetos;
- Maior qualidade, pois os componentes são reutilizados e testados em aplicativos diferentes;
- Facilidade de manutenção/substituição dos componentes.

Riscos associados à CBSE:

- Dificuldade de seleção dos componentes;
- Falta de padrão de interoperabilidade entre componentes;
- Riscos de segurança (uso inadequado do componente);
- Documentação inadequada dos componentes.

Referências

- [1] R. C. Martin.
Clean Architecture: A Craftsman's Guide to Software Structure and Design.
Pearson, 1 edition, 2017.
- [2] R. S. Pressman and B. R. Maxim.
Engenharia de Software uma abordagem profissional, volume 1.
AMGH Editora Ltda, 9. edition, 2021.
- [3] S. R. Schach.
Engenharia de Software: Os paradigmas clássicos & orientados a objetos, volume 1.
McGraw-Hill, 7. edition, 2008.
- [4] I. Sommerville.
Engenharia de Software, volume 1.
São Paulo: Addison-Wesley, 10. edition, 2019.

- [5] M. T. Valente.
Engenharia de software moderna, volume 1.
Independente, 2020.
- [6] R. S. Wazlawick.
Engenharia de Software - Conceitos e Prática, volume 1.
Rio de Janeiro: GEN LTC, 2. edition, 2019.

Perguntas?

Obrigado pela atenção!